



Fundusze
Europejskie
Wiedza Edukacja Rozwój



Rzeczpospolita
Polska

Politechnika
Warszawska

Unia Europejska
Europejski Fundusz Społeczny



Algorytmy w Inżynierii Danych

Optymalizacja drugiego rzędu

Bartosz Chaber

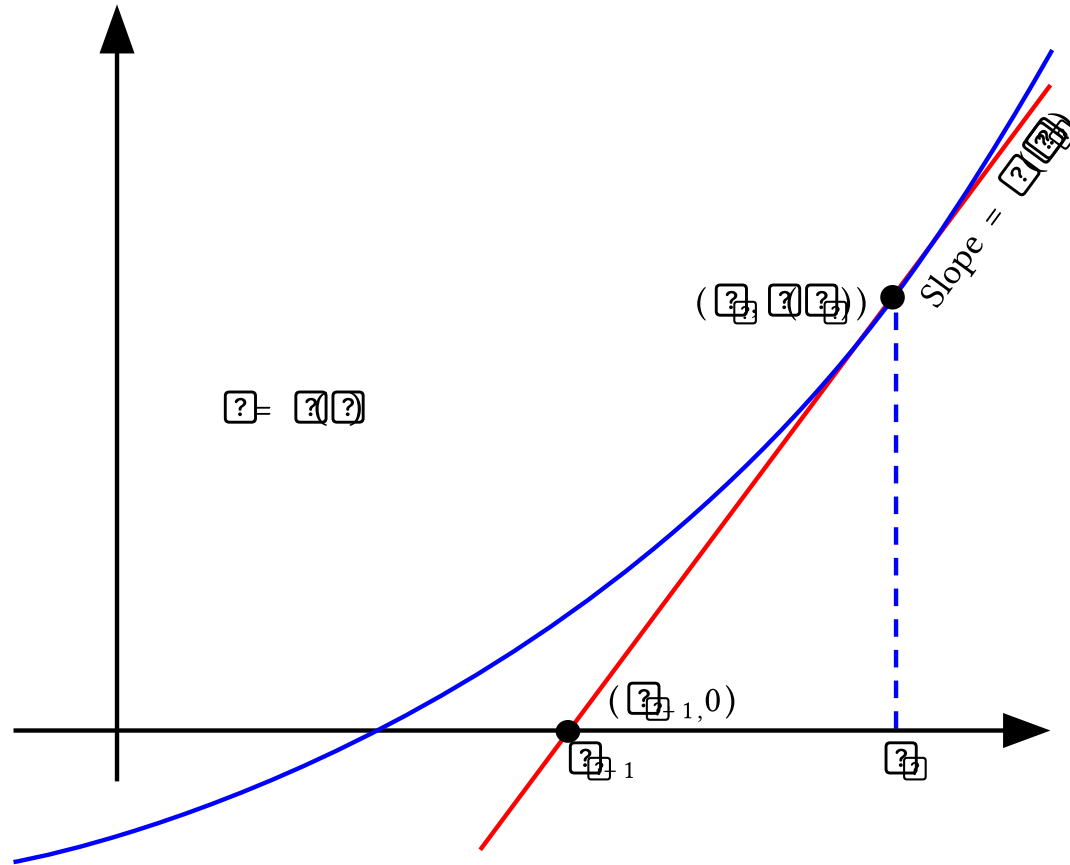
Metody pierwszego rzędu polegają jedynie na kierunku wzrostu wartości minimalizowanej funkcji celu. Na podstawie samego gradientu **nie można** określić optymalnej wartości kroku.

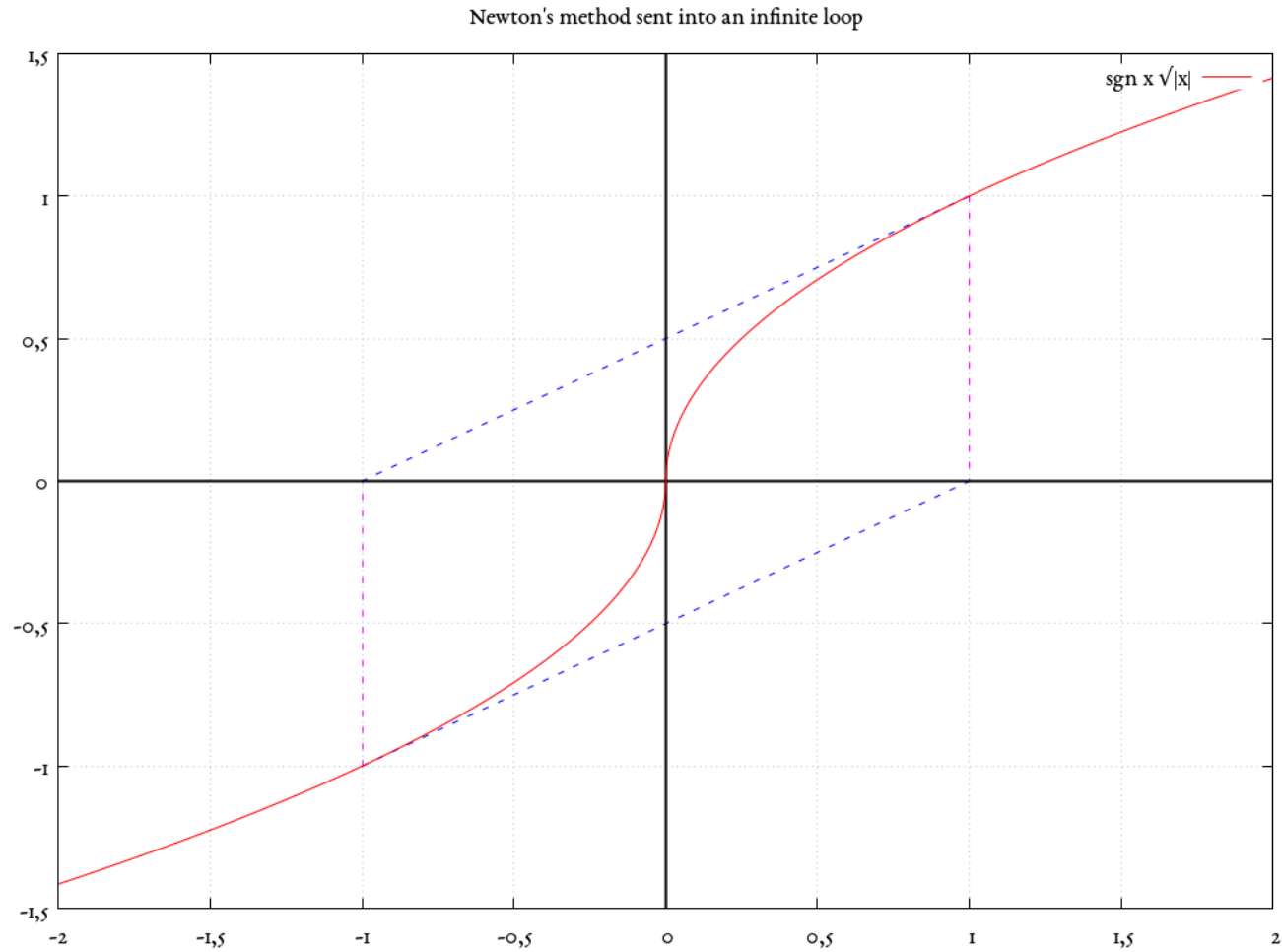
Metoda Newtona dla funkcji jednej zmiennej

Metoda Newtona jest standardową metodą poszukiwania równań nieliniowych $g(x) = 0$. Na podstawie rozwinięcia w szereg Taylora można znaleźć wzór na kolejne przybliżenia: $x_{i+1} = x_i - \frac{g(x_i)}{g'(x_i)}$.

Teraz, zakładając, że poszukujemy minimum funkcji $f(x)$, możemy rozwiązać problem $f'(x) = 0$, czyli wykorzystać wzór:

$$x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)}.$$

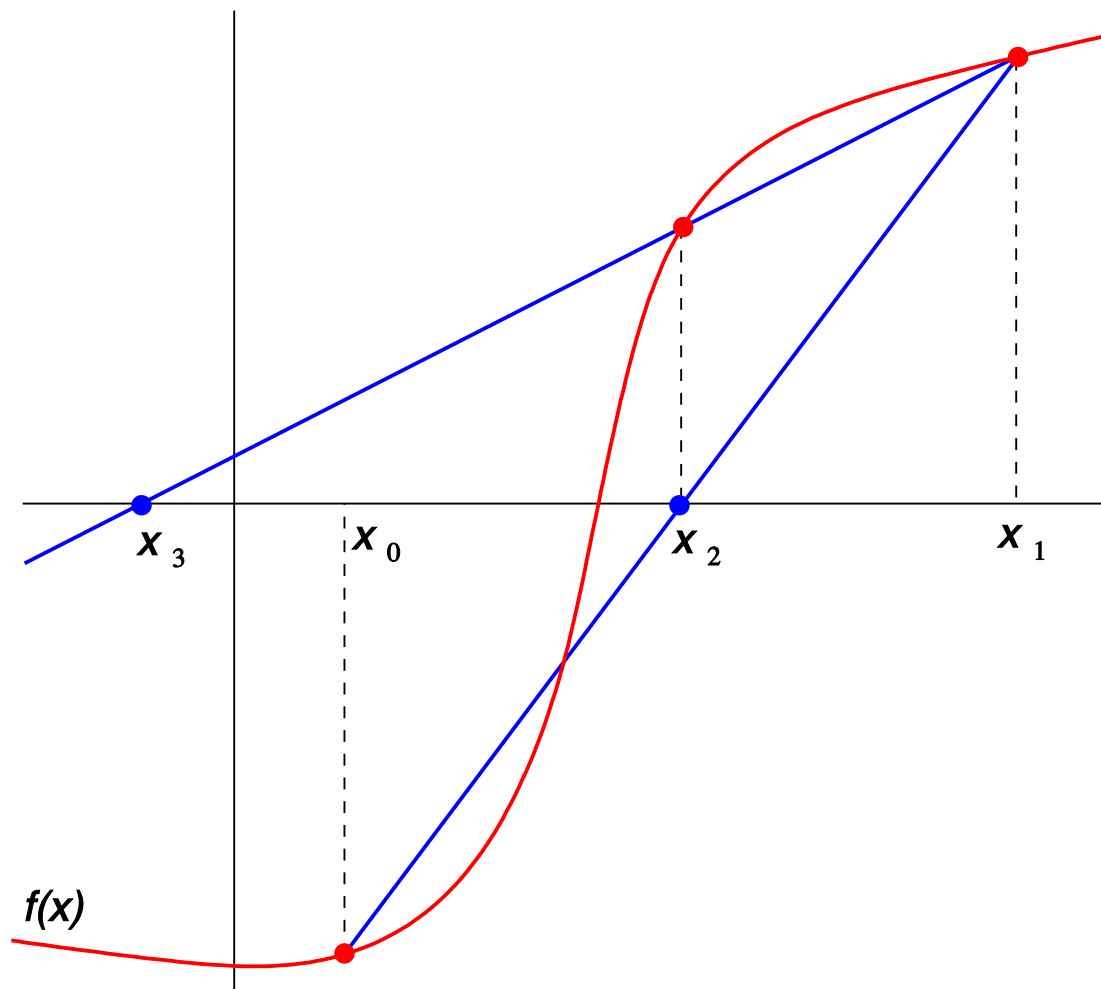




Metoda siecznych dla funkcji jednej zmiennej

Metoda siecznych przybliża nam pochodną funkcji (styczną do wykresu) poprzez sieczną między dwoma punktami.

$$g'(x_i) \approx \frac{g(x_i) - g(x_j)}{x_i - x_j}.$$



Metoda Newtona dla funkcji wielu zmiennych

Poprzednia analiza zakłada, że funkcja f jest funkcją jednej zmiennej x . W przypadku, gdy liczba argumentów funkcji jest większa, zamiast pierwszej pochodnej wykorzystujemy gradient funkcji, a zamiast drugiej pochodnej - macierz Hessego.

$$x_{i+1} = x_i - \frac{\nabla f(x_i)}{\nabla^2 f(x_i)} = x_i - H^{-1}(x_i) \nabla f(x_i),$$

gdzie H^{-1} jest macierzą odwrotną do macierzy Hessego dla funkcji f .

Metoda Newtona

Implementacja w języku Julia może wyglądać następująco (zachowując interfejs DescentMethod):

```
mutable struct NewtonsMethod
    k
    Δ
    NewtonsMethod() = new()
end
```

```
function init!(M::NewtonsMethod, θ)
    M.k = 0
    M.Δ = similar(θ)
end
```

```
function step!(M::NewtonsMethod, ∇f, H, θ;
              ε=1e-8, kmax=10_000)
    k, Δ = M.k, M.Δ
    Δ[:] = H(θ) \ ∇f(θ) # =>  $H^{-1} \cdot \nabla f$ 

    if norm(Δ) < ε || k >= kmax
        return θ
    end

    M.k = k += 1
    return θ - Δ
end
```

Metoda BFGS

Jedną z bolączek metody Newtona jest konieczność znajomości macierzy Hessego. Oczywiście, stosując automatyczne różniczkowanie możemy ją często wyznaczyć, jednak jest to kosztowne (dla p parametrów optymalizowanych macierz Hessego ma wymiar $p \times p$).

Odpowiedzią są metody quasi-Newtona, m.in. BFGS (nazwa pochodzi o nazwisk autorów tej metody: Broyden, Fletcher, Goldfarb, Shanno). W BFGS operuje się gradientem i *przybliżeniem* macierzy Hessego (albo jej odwrotności).



Metoda BFGS

```

using LinearAlgebra
mutable struct BFGS
    Q # approximation of
      # Hessian matrix inverse
    BFGS() = new()
end

function init!(M::BFGS, θ)
    p = length(θ)
    M.Q = Matrix{Float64}(I, p, p)
    return M
end

```

```

function step!(M::BFGS, f, ∇f, θ)
    Q = M.Q
    g = ∇f(θ)
    d = -Q*g
    φ = α -> f(θ + α*d)
    φ' = α -> ∇f(θ + α*d)·d
    α = line_search(φ, φ', d)
    θ' = θ + α*d
    g' = ∇f(θ')
    δ = θ' - θ
    γ = g' - g
    Q[:] = Q - (δ*γ'*Q+Q*γ*δ')/(δ'*γ) +
              (1.0 + (γ'*Q*γ)/(δ'*γ)) *
              (δ*δ')/(δ'*γ)

    return θ'
end

```

Algorytm BFGS wykorzystuje minimalizację kierunkową, poszukując optymalnego kroku α :

```
import Base.MathConstants:  $\phi$ 
function golden_section_search(f, a, b; n=50)
     $\rho = \phi - 1$ 
    d =  $\rho * b + (1 - \rho) * a$ 
    yd = f(d)
    for i = 1:n-1
        c =  $\rho * a + (1 - \rho) * b$ 
        yc = f(c)
        if yc < yd
            b, d, yd = d, c, yc
        else
            a, b = b, c
        end
    end
    return a < b ? (a, b) : (b, a)
end
function line_search( $\phi$ ,  $\phi'$ , d)
    a, b = bracket_minimum( $\phi$ )
    x, y = golden_section_search( $\phi$ , a, b)
     $x/2 + y/2$ 
end
```

```
function bracket_minimum(f;
    x=0, s=1e-2, k=2.0)
    a, ya = x, f(x)
    b, yb = a + s, f(a + s)
    if yb > ya
        a, b = b, a
        ya, yb = yb, ya
        s = -s
    end
    while true
        c, yc = b + s, f(b + s)
        if yc > yb
            return a < c ? (a, c) : (c, a)
        end
        a, ya, b, yb = b, yb, c, yc
        s *= k
    end
end
```

```

# kod na bazie [4]
function zoom( $\phi$ ,  $\phi'$ ,  $\alpha_{lo}$ ,  $\alpha_{hi}$ ;
     $c_1=1e-4$ ,  $c_2=0.1$ ,  $j_{max}=1000$ )
     $\phi'_0 = \phi'(0.0)$ 
    for j=1:jmax
         $\alpha_j = 0.5(\alpha_{lo} + \alpha_{hi})$  # bisection
         $\phi_{\alpha_j} = \phi(\alpha_j)$ 
        if  $\phi_{\alpha_j} > \phi(0.0) + c_1*\alpha_j*\phi'_0$  ||
             $\phi_{\alpha_j} \geq \phi(\alpha_{lo})$ 
             $\alpha_{hi} = \alpha_j$ 
        else
             $\phi'_{\alpha_j} = \phi'(\alpha_j)$ 
            if  $abs(\phi'_{\alpha_j}) \leq -c_2*\phi'_0$ 
                return  $\alpha_j$ 
            end
            if  $\phi'_{\alpha_j}*(\alpha_{hi} - \alpha_{lo}) \geq 0.0$ 
                 $\alpha_{hi} = \alpha_{lo}$ 
            end
             $\alpha_{lo} = \alpha_j$ 
        end
    end
    return  $0.5(\alpha_{lo} + \alpha_{hi})$ 
end

```

```

function line_search( $\phi$ ,  $\phi'$ , d;
     $c_1=1e-4$ ,  $c_2=0.1$ ,  $\rho=0.1$ ,
     $\alpha_{max}=100.$ ,  $j_{max}=1000$ )
     $\alpha_i$ ,  $\alpha_j = 0.0$ ,  $1.0$ 
     $\phi_{\alpha_i}$ ,  $\phi_0$ ,  $\phi'_0 = \phi(\alpha_i)$ ,  $\phi(0.0)$ ,  $\phi'(0.0)$ 
    for j=1:jmax
         $\phi_{\alpha_j} = \phi(\alpha_j)$ 
        if  $\phi_{\alpha_j} > \phi_0 + c_1*\alpha_j*\phi'_0$  ||
            ( $\phi_{\alpha_j} \geq \phi_{\alpha_i}$  &&  $j > 1$ )
            return zoom( $\phi$ ,  $\phi'$ ,  $\alpha_i$ ,  $\alpha_j$ )
        end
         $\phi'_{\alpha_j} = \phi'(\alpha_j)$ 
        if  $abs(\phi'_{\alpha_j}) \leq -c_2*\phi'_0$ 
            return  $\alpha_j$ 
        end
        if  $\phi'_{\alpha_j} \geq 0.0$ 
            return zoom( $\phi$ ,  $\phi'$ ,  $\alpha_j$ ,  $\alpha_i$ )
        end
         $\alpha_i$ ,  $\alpha_j = \alpha_j$ ,  $\rho*\alpha_j + (1.0 - \rho)*\alpha_{max}$ 
         $\phi_{\alpha_i} = \phi_{\alpha_j}$ 
    end
    return  $\alpha_j$ 
end

```

```

function zoom( $\phi$ ,  $\phi'$ ,  $\alpha_{lo}$ ,  $\alpha_{hi}$ ;
     $c_1=1e-4$ ,  $c_2=0.1$ ,  $jmax=1000$ )
 $\phi'_0 = \phi'(0.0)$ 
for  $j=1:jmax$ 
     $\alpha_j = 0.5(\alpha_{lo} + \alpha_{hi})$  # bisection
     $\phi\alpha_j = \phi(\alpha_j)$ 
    if  $\phi\alpha_j > \phi(0.0) + c_1*\alpha_j*\phi'_0$  ||
         $\phi\alpha_j \geq \phi(\alpha_{lo})$ 
         $\alpha_{hi} = \alpha_j$ 
    else
         $\phi'\alpha_j = \phi'(\alpha_j)$ 
        if  $abs(\phi'\alpha_j) \leq -c_2*\phi'_0$ 
            return  $\alpha_j$ 
        end
        if  $\phi'\alpha_j*(\alpha_{hi} - \alpha_{lo}) \geq 0.0$ 
             $\alpha_{hi} = \alpha_{lo}$ 
        end
         $\alpha_{lo} = \alpha_j$ 
    end
end
return  $0.5(\alpha_{lo} + \alpha_{hi})$ 
end

```

Algorithm 3.6 (zoom).**repeat**

Interpolate (using quadratic, cubic, or bisection) to find
a trial step length α_j between α_{lo} and α_{hi} ;

Evaluate $\phi(\alpha_j)$;

if $\phi(\alpha_j) > \phi(0) + c_1\alpha_j\phi'(0)$ or $\phi(\alpha_j) \geq \phi(\alpha_{lo})$

$\alpha_{hi} \leftarrow \alpha_j$;

else

Evaluate $\phi'(\alpha_j)$;

if $|\phi'(\alpha_j)| \leq -c_2\phi'(0)$

Set $\alpha_* \leftarrow \alpha_j$ and **stop**;

if $\phi'(\alpha_j)(\alpha_{hi} - \alpha_{lo}) \geq 0$

$\alpha_{hi} \leftarrow \alpha_{lo}$;

$\alpha_{lo} \leftarrow \alpha_j$;

end (repeat)

Metoda L-BFGS

W przypadku BFGS przechowujemy całą macierz odwrotną do macierzy Hessego.

```
using LinearAlgebra
mutable struct LBFGS
    m :: Int64
    δs :: Vector{Vector{Float64}}
    γs :: Vector{Vector{Float64}}
    qs :: Vector{Vector{Float64}}
    LBFGS() = new()
end
```

```
function init!(M::LBFGS, m)
    M.m = m
    M.δs = Vector{Vector{Float64}}[]
    M.γs = Vector{Vector{Float64}}[]
    M.qs = Vector{Vector{Float64}}[]
    return M
end
```

L-BFGS przechowuje tylko historię m ostatnich gradientów (w `qs`). Następnie wylicza kierunek **bez potrzeby** wyznaczania odwrotności macierzy Hessego.


```

function step!(M::LBFGS, f, ∇f, θ)
    δs, γs, qs = M.δs, M.γs, M.qs
    m, g = length(δs), ∇f(θ)
    d = -g
    if m > 0
        q = g
        for i in m:-1:1
            qs[i] = copy(q)
            q -= (δs[i]·q) / (γs[i]·δs[i]) * γs[i]
        end
        z = (γs[m] .* δs[m] .* q) / (γs[m]·γs[m])
        for i in 1:+1:m
            z += δs[i]*(δs[i]·qs[i]-γs[i]·z)/(γs[i]·δs[i])
        end
        d = -z;
    end
    # ...

```

```

    ϕ = α -> f(θ + α*d)
    ϕ' = α -> ∇f(θ + α*d)·d
    α = line_search(ϕ, ϕ', d)
    θ' = θ + α*d; g' = ∇f(θ')
    δ = θ' - θ; γ = g' - g
    push!(δs, δ)
    push!(γs, γ)
    push!(qs, zero(θ))
    while length(δs) > M.m
        popfirst!(δs)
        popfirst!(γs)
        popfirst!(qs)
    end
    return θ'
end

```

Literatura

- [4] Jorge Nocedal, Stephen Wright, 2006, Numerical Optimization, Springer
- [5] Mykel J. Kochenderfer, Tim A. Wheeler, 2019, Algorithms for Optimization, The MIT Press

Dziękuję za uwagę